

Definindo a Sintaxe De Programas

João Eduardo Montandon
DCC024

Sintaxe X Semântica

- Sintaxe: como os programas aparentam, sua **forma e estrutura**
- Semântica: o que os programas fazem, seu comportamento e significado

Foco da aula: sintaxe!

A Gramática

- Conjunto de regras que define a sintaxe da linguagem
 - A *string* é **válida** quando ela "passa" pela gramática sem inconsistências
-

Um Exemplo

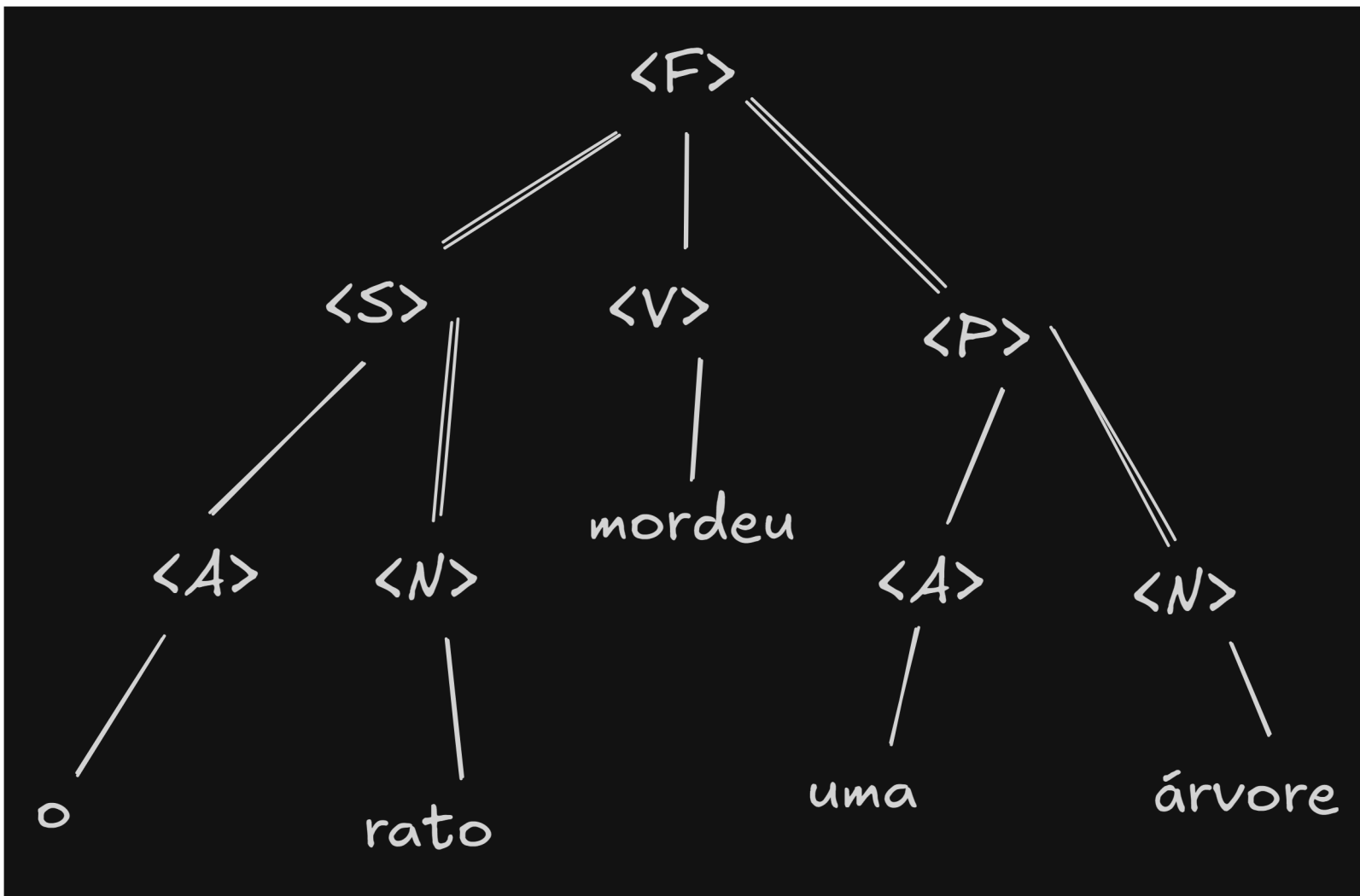
```
<F> ::= <S> <V> <P>  
<S> ::= <A> <N>
```

```
<P> ::= <A> <N> | <PP> <N>  
<V> ::= roeu | mordeu | pulou | comprou  
<A> ::= o | a  
<PP> ::= um | uma | do | da  
<N> ::= rato | pessoa | árvore | carro
```

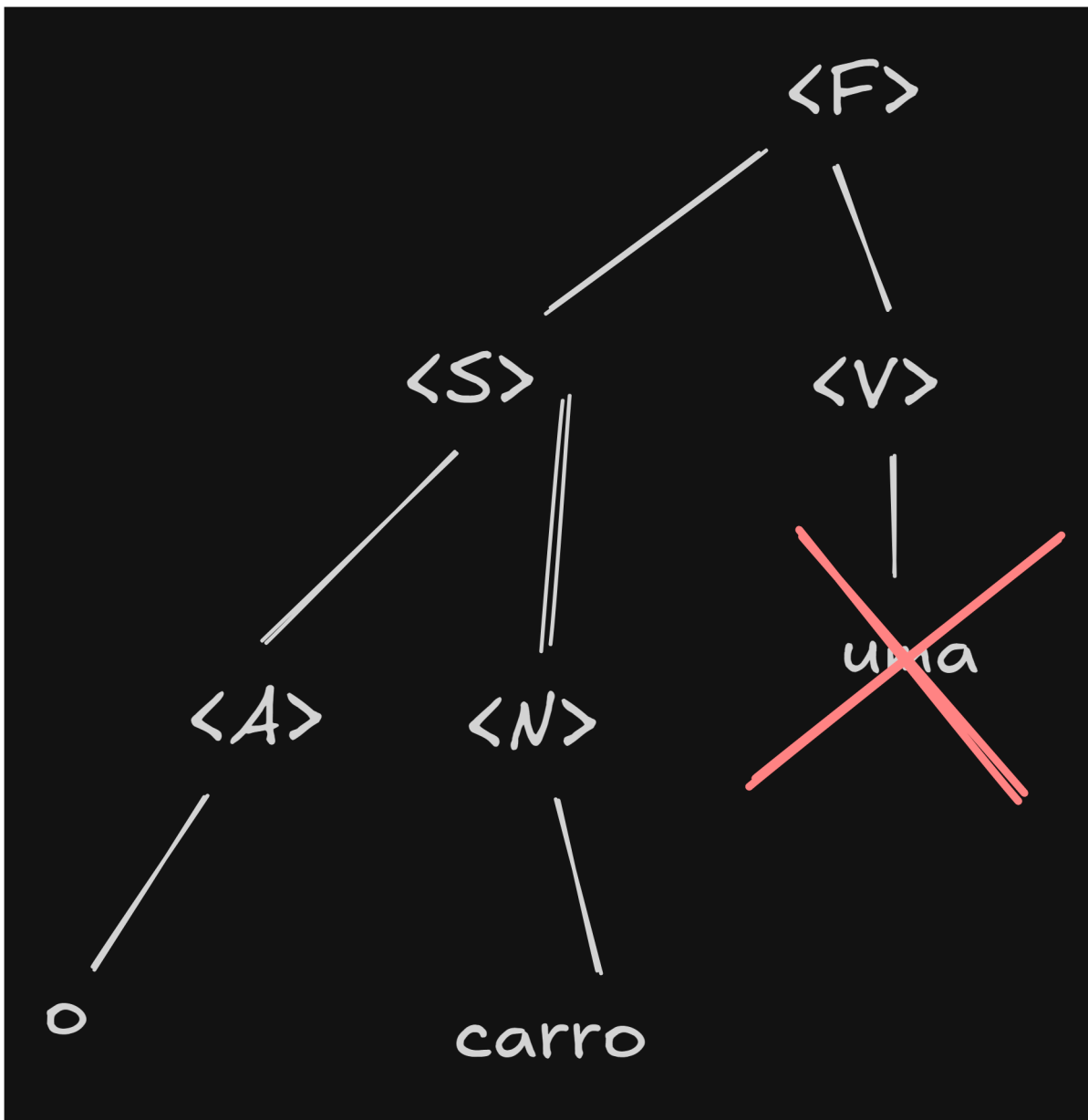
- Uma frase é um sujeito, verbo e predicado
 - Um sujeito é um artigo e um nome
 - Um predicado é um artigo e nome, ou preposição e nome
-

Como Funciona?

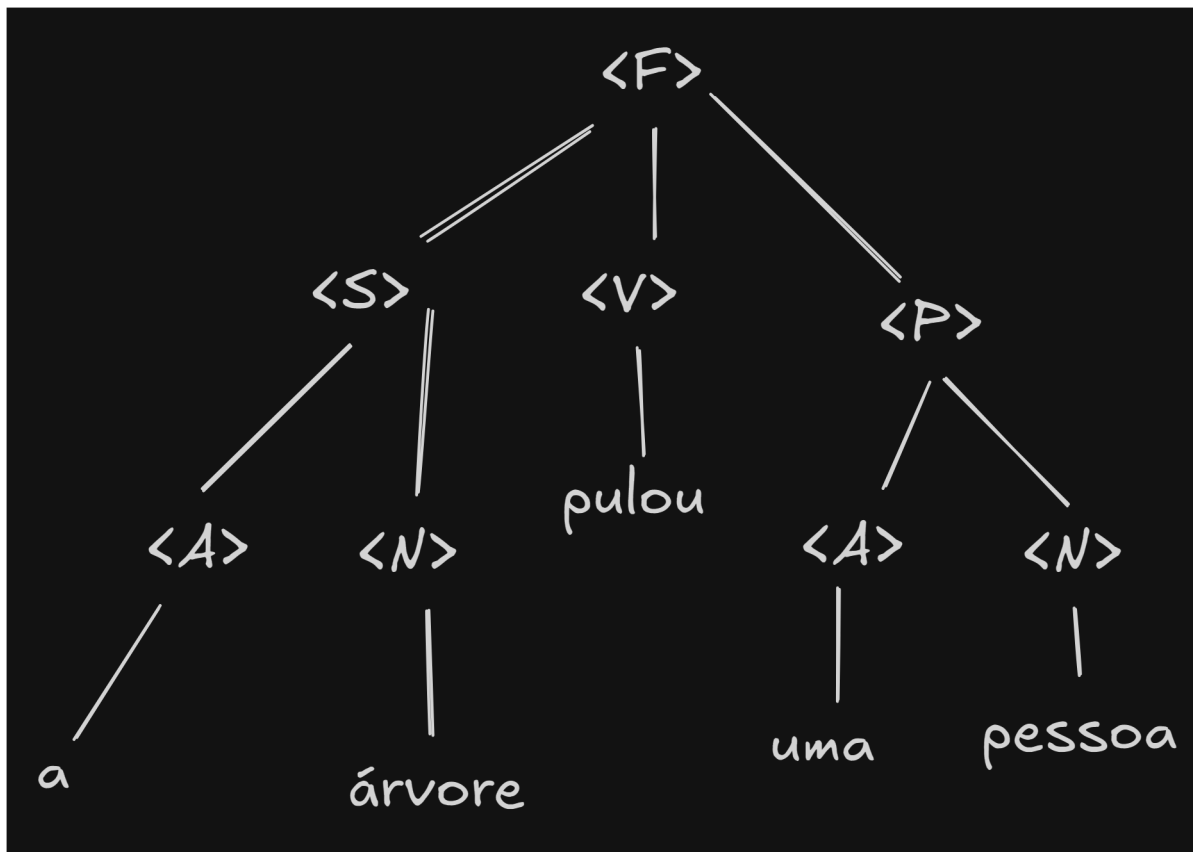
- As regras determinam como uma árvore é construída (**árvore de parsing**)
 - O símbolo inicial fica na raiz da árvore
 - Os demais nós são gerados de acordo com o texto da entrada
 - Válido quando uma árvore é gerada para o texto
 - Inválido quando essa árvore não pode ser gerada
-



o rato mordeu uma árvore ✓



o carro uma pessoa comprou ✖

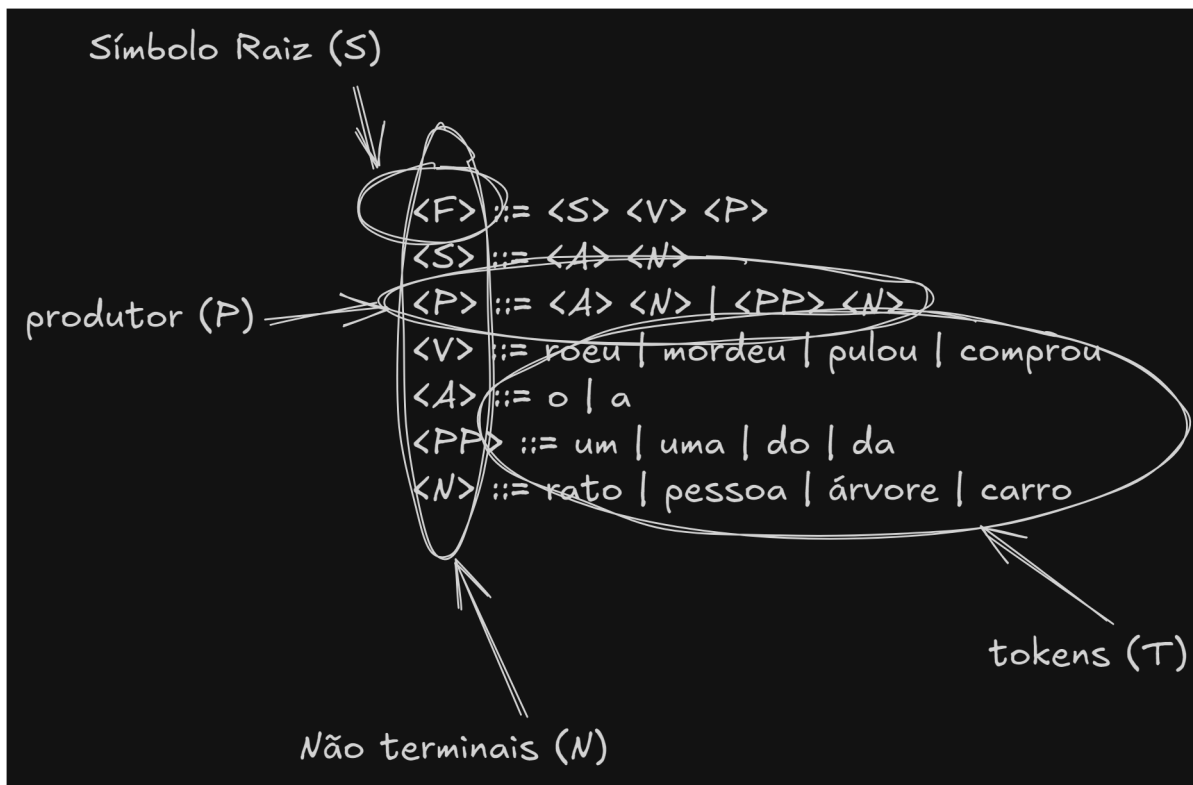


a árvore pulou uma pessoa

- Está correta? ✓
- Faz sentido? 🤔 (Semântica...)

Gramáticas BNF

Representada por <T, N, P, S>



- Tokens (T): a menor unidade de sintaxe da gramática
 - Caracteres, literalmente
 - São atômicas, indivisíveis
- Não Terminais (N): representam peças "maiores" da sintaxe
 - Representados por $\langle R \rangle$
 - Nós intermediários da árvore
- Produtores (P): regras que levam a expansão da árvore
- Símbolo Raiz (S): Símbolo inicial da gramática

- Toda regra é uma atribuição definida por `::=`
 - A esquerda fica o símbolo não terminal
 - A direita fica os símbolos a serem derivados para aquela regra
 - Utilize `|` para expressar diferentes derivações para o mesmo símbolo
-

A Árvore De Parsing

1. Coloque o símbolo na raiz
 2. Adicione filhos a todo nó não-terminal, recursivamente
 3. Finalizado quando todas folhas forem tokens
 4. Ler as folhas da esquerda para a direita para obter o texto derivado
-

Outro Exemplo

```
<exp> ::= <exp> "+" <exp>
        | <exp> "*" <exp>
        | "(" <exp> ")"
        | "a"
        | "b"
        | "c"
```

Quais seriam as árvores para...

- a + b

- $(a + b)$
-

O que acabamos de fazer? **Parsing**

Quais seriam as árvores para...

- $(a + (b))$
 - $a + b * c$ 🧐🧐
-

$a + b * c$

Uma gramática é **ambígua** quando duas ou mais árvores são derivadas a partir da mesma entrada

Por que isso importa?

Construindo Gramáticas

- É como programar
 - Programa: imaginar como o código levará ao resultado esperado
 - Gramática: imaginar como a gramática derivará a árvore desejada
 - Então, **divida para conquistar!**
-

Exemplo

```
float a;  
boolean a,b,c;  
int a=1, b, c=1+2;
```

(1) Focar apenas na declaração do tipo

```
<declaracao-variavel> ::= <tipo> <lista-declaracao>;  
<tipo> ::= boolean  
          | byte  
          | short  
          | int  
          | double  
          | long  
          | char  
          | float
```

(2) Expandir para declaração de variáveis, apenas

```
<lista-declaracao> ::= <declaracao>  
                      | <declaracao> , <lista-declaracao>;
```

(3) Adicionar inicialização

```
<declaracao> ::= <nome-variavel>  
                | <nome-variavel> = <expressao>
```

```
<declaracao-variavel> ::= <tipo> <lista-declaracao>;  
<tipo> ::= boolean | byte | short | int  
          | double | long | char | float  
<lista-declaracao> ::= <declaracao>  
                        | <declaracao> , <lista-declaracao>;  
<declaracao> ::= <nome-variavel>  
                  | <nome-variavel> = <expressao>
```

Como fazer para testar?

Estrutura Léxica E Frasal

```
<exp>    ::= <exp> "+" <exp>  
            | <exp> "*" <exp>  
            | "(" <exp> ")"  
            | "a"  
            | "b"  
            | "c"
```

1. (a + b)
2. (a+b)
3. (a + b)

Geram a mesma árvore, mas são strings diferentes... por que? 🤔

Selecionamos os tokens, arbitrariamente, como isso é feito pela máquina?

- **Estrutura frasal:** construir árvores a partir de uma lista de tokens
 - **Estrutura léxica:** obter tokens a partir de uma string
-

Antigamente

Ambas gramáticas eram especificadas em conjunto

```
<if-stmt> ::= if <white-space> <expr> <white-space>  
           then <white-space>  
               <stmt> <white-space> <else-part>  
<else-part> ::= else <white-space> <stmt> | <empty>
```

O que acharam? 🤖

Hoje

Gramáticas separadas

- Uma para obter a sequência de tokens (scanner)

- Uma para construir a árvore de parsing (parser)

```
<program-file> ::= <end-of-file> | <element> <program-file>  
<element> ::= <token> | <one-white-space> | <comment>  
<one-white-space> ::= <space> | <tab> | <end-of-line>  
<token> ::= <identifier> | <operator> | <constant> | ...
```

Outras Notações

- Variações BNF
- Variações EBNF
- Diagramas de Sintaxe

BNF

1. Algumas usam `-->` ao invés de `::=`
2. Algumas usam negrito/itálico no lugar de `<>`
3. Aspas simples para representar símbolos como tokens (ex: `'|'`)

EBNF

Sintaxe estendida para aumentar expressividade da gramática

1. `{x}` zero ou mais ocorrências de `x`

2. `[x]` ocorrência de `x` é opcional
3. `()` para agrupamentos
4. `|` alternativas de expansão (já vimos)
5. `""` para distinguir tokens de símbolos

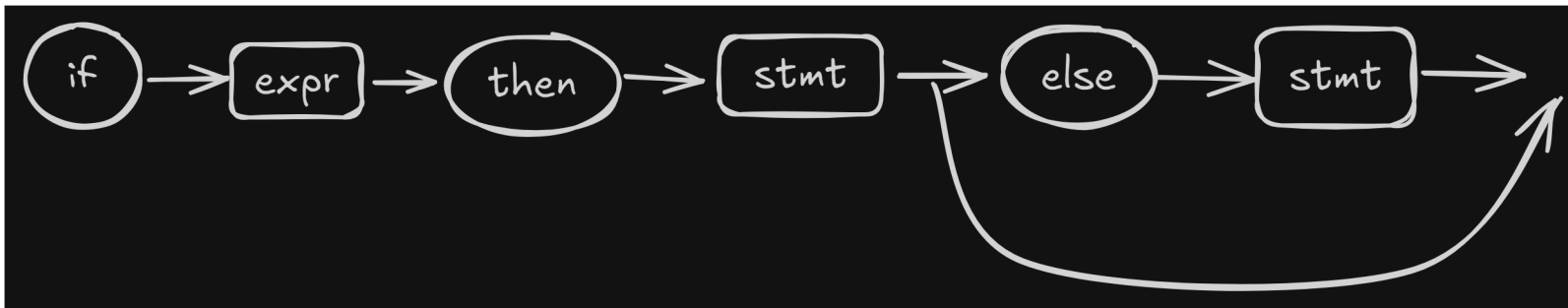
```
<if-stmt> ::= if <expr> then <stmt> [else <stmt>]
```

```
<thing-list> ::= { (<stmt> | <declaration>) ; }
```

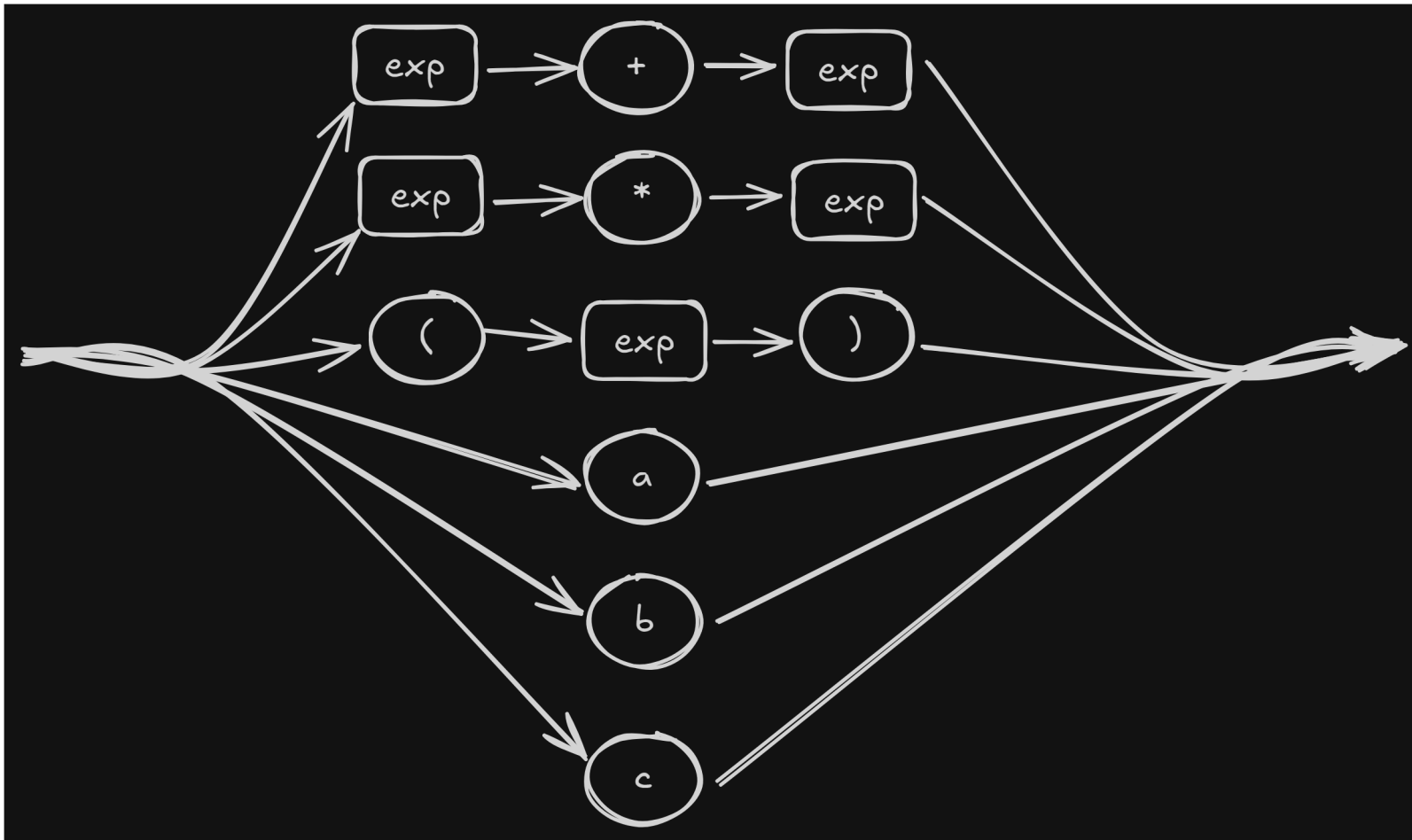
Diagramas De Sintaxe

- Representação visual de gramáticas EBNF
- Sequência de formas conectadas sequencialmente
 - retângulos: não terminais
 - ovais: terminais

```
<if-stmt> ::= if <expr> then <stmt> [else <stmt>]
```



`<exp> ::= <exp> + <exp> | <exp> * <exp> | (<exp>) | a | b | c`



Prós E Contras

- Ótima opção para se ter uma "ideia" da gramática (Big Picture)
 - Difícil de se ler precisamente (humanos e máquinas)
 - Qual é a árvore de parsing?
-

Gramáticas Livres De Contexto

- Parsing pode ser representado por gramáticas livres de contexto

```
S --> aSb | X
```

```
X --> cX | \0
```

- Outras opções podem ser adotadas
 - Gramáticas regulares para gramáticas léxicas
-

Conclusão

- Gramáticas são utilizadas para definir a sintaxe de uma linguagem
 - Gramática léxica: **texto** --> **tokens**
 - Gramática frasal: **tokens** --> **árvore de parsing**
- Conexão entre teoria e prática
 - Padronização das gramáticas com BNF
 - Simplificação das gramáticas em linguagens modernas